

Задание 5 ЕГЭ. Итоговое повторение

11 класс Информатика ЕГЭ



Теория: инструменты Python

Функция `bin()`: число $\rightarrow$ двоичная строка

Синтаксис

```
1 N = 13
2 s = bin(N)      # '0b1101' -- с префиксом
3 s = bin(N)[2:]  # '1101'   -- без префикса
4
```

Примеры

```
1 bin(5)[2:]      # '101'
2 bin(8)[2:]      # '1000'
3 bin(1)[2:]      # '1'
4 bin(255)[2:]    # '11111111'
5
```

Функция `int()`: двоичная строка → число

Синтаксис

```
1 s = '1101'
2 R = int(s, 2)    # 13
3
```

Второй аргумент — основание системы счисления.

Примеры

```
1 int('101', 2)    # 5
2 int('1000', 2)   # 8
3 int('110', 3)    # 12    троичная(!)
4 int('13', 4)     # 7     четверичная(!)
5
```

Функция `to_base()`: перевод в любую систему

Проблема

В Python нет готовых функций для троичной, четверичной и других систем — пишем свою.

Код функции

```
1 def to_base(n, base):  
2     s = ''  
3     while n > 0:  
4         s = str(n % base) + s  
5         n //= base  
6     return s  
7
```

Функция `to_base()`: примеры

Примеры

```
1 to_base(11, 3) # '102'
2 to_base(12, 3) # '110'
3 to_base(9, 3)  # '100'
4 to_base(7, 4)  # '13'
5
```

Обратный перевод

```
1 int('102', 3) # 11
2 int('110', 3) # 12
3 int('13', 4)  # 7
4
```

Дописывание цифр к строке

Справа (конкатенация)

```
1 s = '101'
2 s = s + '10'      # '10110'
3 s += '0'          # '1011'
4 s += s[-3:]        # копия хвоста
5
```

С обеих сторон

```
1 s = '101'
2 s = '1' + s + '01'
3     # '110101'
4
```

Слева

```
1 s = '101'
2 s = '1' + s      # '1101'
3 s = '10' + s     # '10101'
4
```

Пример из задачи

«Слева 10, справа 01»:

```
1 s = '10' + s + '01'
2
```

Срезы строк и их применение

Срезы и типичные приёмы

```
1 s = '110100'
2 s[:2]          # '11'   -- первые 2
3 s[2:]          # '0100' -- без первых 2
4 s[-3:]         # '100'  -- последние 3
5
6 # замена первых 2 разрядов на '10':
7 s = '10' + s[2:]
8
9 # дописать 3 последние цифры:
10 s += s[-3:]
11
```


Замена символов и подсчёт

replace() — замена

```
1 s = '100'
2 s.replace('1', '11')
3   # '1100'
4 s.replace('0', '00')
5   # '10000'
6
```

Заменяет **каждое** вхождение.

count() — подсчёт

```
1 s = '11010'
2 s.count('1')      # 3
3 s.count('1') % 2  # 1
4
```

Сумма цифр (троичная)

```
1 s = '1201'
2 sum(int(c) for c in s)
3   # 4
4
```

Проверка условий

Чётность и делимость

```
1 N % 2 == 0    # N чётное?  
2 N % 2 != 0    # N нечётное?  
3 N % 3 == 0    # N делится на 3?  
4
```

Остаток от деления

```
1 N = 7  
2 N % 3          # 1  
3 (N % 3) * 5    # 5  
4 bin(5)[2:]     # '101'  
5
```

Шаблон: ищем минимальное

break — останавливаемся на первом

```
1 for N in range(1, 1000):
2     s = bin(N)[2:]
3
4     # ... правила алгоритма ...
5
6     R = int(s, 2)
7     if R > 253:
8         print(N)
9         break
10
```

Когда использовать

Если нужно **минимальное** N или **первый подходящий** — перебираем N по порядку и делаем break.

Шаблон: ищем максимальное / мин. среди R

Список + `max()` / `min()`

```
1 res = []
2 for N in range(1, 1000):
3     s = bin(N)[2:]
4     # ... правила алгоритма ...
5     R = int(s, 2)
6     if R <= 162:
7         res.append(R)
8
9 print(max(res))
10
```

Когда использовать

Макс./мин. N или $R \rightarrow$ собираем в список, берём `max()` / `min()`.



Классная работа

Задача 1

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится двоичная запись числа N .
2. К этой записи дописываются справа ещё два разряда по следующему правилу:
 - а) складываются все цифры двоичной записи, и остаток от деления суммы на 2 дописывается в конец числа (справа);
 - б) над этой записью производятся те же действия — справа дописывается остаток от деления суммы её цифр на 2.

Полученная запись является двоичной записью числа R .

Например, $12_{10} = 1100_2 \rightarrow 110000_2 = 48_{10}$, а $7_{10} = 111_2 \rightarrow 11110_2 = 30_{10}$.

Укажите такое наименьшее число N , для которого результат работы алгоритма больше числа **253**.

Задача 1: Решение

```
1  for N in range(1, 1000):
2      s = bin(N)[2:]
3
4      bit1 = s.count('1') % 2    # первый бит чётности
5      s += str(bit1)
6
7      bit2 = s.count('1') % 2    # второй бит чётности
8      s += str(bit2)
9
10     R = int(s, 2)
11     if R > 253:
12         print(N)
13         break
14
```

Задача 2

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится двоичная запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если сумма цифр в двоичной записи числа чётная, то к записи справа дописывается **0**, а затем два левых разряда заменяются на **10**;
 - б) если сумма цифр нечётная, то к записи справа дописывается **1**, а затем два левых разряда заменяются на **11**.

Полученная запись является двоичной записью числа R .

Например, $6_{10} = 110_2 \rightarrow 1000_2 = 8_{10}$, а $4_{10} = 100_2 \rightarrow 1101_2 = 13_{10}$.

Укажите минимальное число N , после обработки которого получается число R , большее **29**.

Задача 2: Решение

```
1  for N in range(1, 1000):
2      s = bin(N)[2:]
3
4      if s.count('1') % 2 == 0:      # сумма цифр чётная
5          s += '0'
6          s = '10' + s[2:]
7      else:                          # сумма цифр нечётная
8          s += '1'
9          s = '11' + s[2:]
10
11     R = int(s, 2)
12     if R > 29:
13         print(N)
14         break
15
```

Задача 3

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится двоичная запись числа N .
2. К этой записи дописываются справа ещё несколько разрядов по следующему правилу:
 - а) если N чётное, то к нему справа приписываются два нуля, а слева единица;
 - б) если N нечётное, то к нему справа приписывается в двоичном виде сумма цифр его двоичной записи.

Полученная запись является двоичной записью числа R .

Например, $4_{10} = 100_2 \rightarrow 110000_2 = 48_{10}$, а $13_{10} = 1101_2 \rightarrow 110111_2 = 55_{10}$.

Укажите такое число N , для которого число R является наименьшим среди чисел, превышающих **190**.

Задача 3: Решение

```
1 results = []
2 for N in range(1, 1000):
3     s = bin(N)[2:]
4
5     if N % 2 == 0:                # чётное
6         s = '1' + s + '00'
7     else:                        # нечётное
8         d = s.count('1')         # сумма цифр
9         s += bin(d)[2:]          # справа в двоичном виде
10
11    R = int(s, 2)
12    if R > 190:
13        results.append((R, N))
14
15 results.sort()
16 print(results[0][1])             # N для мин. R
17
```

Задача 4

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится двоичная запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если число чётное, то к двоичной записи числа слева дописывается **10**;
 - б) если число нечётное, то к двоичной записи числа слева дописывается **1** и справа дописывается **01**.

Полученная запись является двоичной записью числа R .

Результат переводится в десятичную систему и выводится на экран.

Например, $4_{10} = 100_2 \rightarrow 10100_2 = 20_{10}$, а $5_{10} = 101_2 \rightarrow 110101_2 = 53_{10}$.

Укажите максимальное число R , которое может быть результатом работы данного алгоритма, при условии, что N не больше **12**.

Задача 4: Решение

```
1 max_R = 0
2 for N in range(1, 13):           # N от 1 до 12
3     s = bin(N)[2:]
4
5     if N % 2 == 0:               # чётное
6         s = '10' + s
7     else:                        # нечётное
8         s = '1' + s + '01'
9
10    R = int(s, 2)
11    if R > max_R:
12        max_R = R
13
14 print(max_R)
15
```

Задача 5

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится двоичная запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если число N делится на 3, то к этой записи дописываются три последние двоичные цифры;
 - б) если число N на 3 не делится, то остаток от деления умножается на 3, переводится в двоичную запись и дописывается в конец числа.

Полученная запись является двоичной записью числа R .

Например, $12_{10} = 1100_2 \rightarrow 1100100_2 = 100_{10}$, а $4_{10} = 100_2 \rightarrow 100011_2 = 35_{10}$.

Укажите максимальное число R , не превышающее **162**, которое может быть получено с помощью описанного алгоритма.

Задача 5: Решение

```
1 res = []
2 for N in range(1, 1000):
3     s = bin(N)[2:]
4
5     if N % 3 == 0:                # делится на 3
6         s += s[-3:]
7     else:                        # не делится на 3
8         rem = (N % 3) * 3
9         s += bin(rem)[2:]
10
11    R = int(s, 2)
12    if R <= 162:
13        res.append(R)
14
15 print(max(res))
16
```

Задача 6

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится двоичная запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если число N делится на 3, то к этой записи дописываются её три последние двоичные цифры;
 - б) если число N на 3 не делится, то остаток от деления умножается на 3, переводится в двоичную запись и дописывается в конец числа.

Полученная запись является двоичной записью числа R .

Например, $12_{10} = 1100_2 \rightarrow 1100100_2 = 100_{10}$, а $4_{10} = 100_2 \rightarrow 100011_2 = 35_{10}$.

Укажите минимальное число N , после обработки которого с помощью этого алгоритма получается число R , не меньшее **200**.

Задача 6: Решение

```
1  for N in range(1, 1000):
2      s = bin(N)[2:]
3
4      if N % 3 == 0:                # делится на 3
5          s += s[-3:]
6      else:                        # не делится на 3
7          rem = (N % 3) * 3
8          s += bin(rem)[2:]
9
10     R = int(s, 2)
11     if R >= 200:
12         print(N)
13         break
14
```

Задача 7

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится двоичная запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если число N чётное, то в этой записи каждая единица заменяется на **11**;
 - б) если число N нечётное, то в этой записи каждый нуль заменяется на **00**.

Полученная запись является двоичной записью числа R .

Результат переводится в десятичную систему и выводится на экран.

Например, $4_{10} = 100_2 \rightarrow 1100_2 = 12_{10}$, а $5_{10} = 101_2 \rightarrow 1001_2 = 9_{10}$.

Укажите минимальное число N , после обработки которого с помощью этого алгоритма получается число R , большее **130**.

Задача 7: Решение

```
1  for N in range(1, 1000):
2      s = bin(N)[2:]
3
4      if N % 2 == 0:                # чётное
5          s = s.replace('1', '11')
6      else:                        # нечётное
7          s = s.replace('0', '00')
8
9      R = int(s, 2)
10     if R > 130:
11         print(N)
12         break
13
```

Задача 8

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится **троичная** запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если число N делится на 3, то к этой записи дописываются две последние **троичные** цифры;
 - б) если число N на 3 не делится, то остаток от деления умножается на 5, переводится в троичную запись и дописывается в конец числа.

Полученная запись является **троичной** записью числа R .

Например, $11_{10} = 102_3 \rightarrow 102101_3 = 307_{10}$, а $12_{10} = 110_3 \rightarrow 11010_3 = 111_{10}$.

Укажите минимальное число R , большее **133**, которое может быть получено с помощью описанного алгоритма.

Задача 8: Решение

```
1 def to3(n):
2     s = ''
3     while n > 0:
4         s = str(n % 3) + s
5         n //= 3
6     return s
7
8 res = []
9 for N in range(1, 1000):
10     s = to3(N)
11
12     if N % 3 == 0:                # делится на 3
13         s += s[-2:]
14     else:                        # не делится
15         s += to3((N % 3) * 5)
16
17     R = int(s, 3)
18     if R > 133:
19         res.append(R)
20
21 print(min(res))
22
```

Задача 9

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится **троичная** запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если число N делится на 3, то к этой записи дописываются две последние **троичные** цифры;
 - б) если число N на 3 не делится, то остаток от деления умножается на 3, переводится в троичную запись и дописывается в конец числа.

Полученная запись является **троичной** записью числа R .

Например, $6_{10} = 20_3 \rightarrow 2020_3 = 60_{10}$, а $4_{10} = 11_3 \rightarrow 1110_3 = 39_{10}$.

Укажите максимальное число N , после обработки которого с помощью этого алгоритма получается число, не превышающее **150**.

Задача 9: Решение

```
1 def to3(n):
2     s = ''
3     while n > 0:
4         s = str(n % 3) + s
5         n //= 3
6     return s
7
8 res = []
9 for N in range(1, 1000):
10    s = to3(N)
11
12    if N % 3 == 0:                # делится на 3
13        s += s[-2:]
14    else:                        # не делится
15        s += to3((N % 3) * 3)
16
17    R = int(s, 3)
18    if R <= 150:
19        res.append(N)
20
21 print(max(res))
22
```

Задача 10

Задача

На вход алгоритма подаётся натуральное число N . Алгоритм строит по нему новое число R следующим образом.

1. Строится **троичная** запись числа N .
2. Далее эта запись обрабатывается по следующему правилу:
 - а) если число N делится на 3, то к этой записи справа дописываются две последние **троичные** цифры;
 - б) если число N на 3 не делится, то вычисляется сумма цифр полученной троичной записи, эта сумма умножается на 3, переводится в троичную систему и дописывается в конец числа.

Полученная запись является **троичной** записью числа R .

Например, $8_{10} = 22_3 \rightarrow 22110_3 = 228_{10}$, а $9_{10} = 100_3 \rightarrow 10000_3 = 81_{10}$.

Укажите минимальное **чётное** число R , большее **155**, которое может быть получено с помощью описанного алгоритма.

Задача 10: Решение

```
1 def to3(n):
2     s = ''
3     while n > 0:
4         s = str(n % 3) + s
5         n //= 3
6     return s
7
8 res = []
9 for N in range(1, 1000):
10    s = to3(N)
11
12    if N % 3 == 0:                # делится на 3
13        s += s[-2:]
14    else:                        # не делится
15        d = sum(int(c) for c in s) # сумма цифр
16        s += to3(d * 3)
17
18    R = int(s, 3)
19    if R > 155 and R % 2 == 0:    # чётное R > 155
20        res.append(R)
21
22 print(min(res))
```



Ответы

Ответы

№	Что ищем	Ответ
1	Наименьшее N , при котором $R > 253$	64
2	Минимальное N , при котором $R > 29$	11
3	N , дающее минимальное $R > 190$	16
4	Максимальное R при $N \leq 12$	109
5	Максимальное $R \leq 162$	151
6	Минимальное N , при котором $R \geq 200$	26
7	Минимальное N , при котором $R > 130$	22
8	Минимальное $R > 133$ (троичная)	141
9	Максимальное N , при котором $R \leq 150$ (троичная)	16
10	Минимальное чётное $R > 155$ (троичная)	162

Спасибо за внимание!